Contents

Goodreads Books — Module 3 Lab Guide

Scenario: Books & publishing. Predict a book's Goodreads rating before publication. **Module:** 3 (Data Preparation and Feature Engineering) **Audience:** Adult learners, A2 English. New to AI. **Tools:** Google Colab (no install needed).

The Scenario — Your Mission

Where you work

You and your team are new data analysts at a **publishing house**. The company prints books and sells them in shops and online.

Every year: - **1,000 manuscript submissions** arrive from writers. - The editorial team picks only **50** to publish. - A bad pick costs the company **$200,000** in printing + marketing for a book that does not sell.

The problem

The team picks 50 books out of 1,000. But they often pick wrong. A book that the editor loved gets 2 stars on Goodreads and nobody buys it.

Ravi is worried. He calls your team into a meeting.

Your manager's request

Your manager, **Ravi** (VP of Editorial), tells you:

> "I cannot read 1,000 manuscripts. My team cannot either.
>
> I need a different tool. Read the proposed book's metadata: title, authors, page count, language, description. Predict the rating it will get on Goodreads.
>
> We will publish only books your model says will rate above 4.0.
>
> If we save 5 wrong picks per year, we save $1 million."

Your team's job for the next 2 weeks (Module 3)

Ravi sends you a clean CSV with **11,000 books** from Goodreads. Each row has the title, authors, publisher, language, page count, ratings, and reviews.

Your job in Module 3: > **Turn this CSV into ONE clean file. The clean file will be used to train the model in Module 4.**

The clean file is called `goodreads_books_clean.parquet`. It must have **21 specific columns** (we will see them in Class 6).

After Module 3

Module	What happens
Module 4	Train the rating model. Ravi gets his "expected rating" tool.
Module 5	Find groups of similar books. Useful for genre marketing.
Module 7	Use the book description text. Predict genre or rating from description alone.

You use the **same Goodreads dataset** until the end of Module 7.

How This Guide Works (Read This First!)

We do NOT give you the full code

In this guide you will see:

1. WHAT / WHY / EXPECTED — explained in words

For every step: - **WHAT** you have to do - **WHY** you do it - **HINTS** — function names + arguments - **EXPECTED** — the result you should see

2. Sometimes a code skeleton with blanks

For harder steps you get a skeleton with ___ blanks. **You fill in the blanks.**

Why no full code?

You will NOT learn if you copy-paste. Writing the code yourself takes 10 minutes more today, but saves 10 hours later.

Rule: If you finish a step in 30 seconds by copy-pasting from a teammate, redo it yourself with different variable names.

Visualizations — Two Modes

In every class you will make charts.

1. Exploratory charts (for YOU)

Fast, ugly, no labels. Goal: "What does this data look like?"

2. Explanatory charts (for RAVI)

Clean, labeled, one clear message. Goal: "Ravi, look at this. This is the problem."

In every class you make BOTH kinds.

Your plotting toolkit

Plot	When to use it	Function name
Histogram	Shape of a numeric column	`plt.hist()`
Bar chart	Count of each category	`df['col'].value_counts().plot.bar()`
Box plot	Outliers in a numeric column	`sns.boxplot()`
Scatter plot	Relationship between 2 numbers	`plt.scatter()`
Heatmap	Correlation between many columns	`sns.heatmap(df.corr())`
Line plot	Change over time	`plt.plot()`

Before you start: `import matplotlib.pyplot as plt` and `import seaborn as sns`.

Before You Start — Google Colab Setup

Step 1 — Open a new Colab notebook

1. Go to https://colab.research.google.com
2. Sign in with your Google account.
3. Click **"New notebook"**.
4. Name it `goodreads_module_3.ipynb`.

Step 2 — Connect to Google Drive

```
from google.colab import drive
drive.mount('/content/drive')
```

Step 3 — Create a project folder

```
import os
os.makedirs('/content/drive/MyDrive/goodreads_lab', exist_ok=True)
%cd /content/drive/MyDrive/goodreads_lab
```

Step 4 — Get the data

Option A — Direct from Kaggle:

1. Free Kaggle account.
2. Get a Kaggle API token (`kaggle.json`).
3. In Colab:

```
from google.colab import files
files.upload()    # pick kaggle.json
```

4. Then:

```
!mkdir -p ~/.kaggle && mv kaggle.json ~/.kaggle/ && chmod 600 ~/.kaggle/kaggle.json
!pip install kaggle -q
!kaggle datasets download -d jealousleopard/goodreadsbooks
!unzip -q goodreadsbooks.zip -d data
!ls data/
```

Option B — Upload by hand in Colab's file panel.

Step 5 — Test it

```
import pandas as pd
df = pd.read_csv('data/books.csv', on_bad_lines='skip')
print(df.shape)
df.head()
```

Should print about `(11000, 12)`. The `on_bad_lines='skip'` is needed because this CSV has some broken rows.

Colab tips

Tip	Why
Save your notebook to Drive often	Colab disconnects after 12 hours.
Save intermediate `.parquet` files to Drive	Files in `/content/` disappear when Colab disconnects.
Share the notebook with teammates (Share button, top right)	Editor access. Like Google Docs for code.

Class 1 — Data Cleaning

> **Scenario reminder:** Ravi drops a CSV on your desk. Some rows are broken. The publication date is a messy string. Some books have 0 pages. Your job today: clean up.

Your goal

Sample, fix dates, deal with broken rows, decide what to keep.

Inputs

- `data/books.csv`

Outputs

- `books_step1.parquet` saved in your Drive folder
- 3+ exploratory charts
- 1 explanatory chart for Ravi
- Notes in markdown

Phase A — Explore the data first (15 min)

Exploratory chart 1 — Distribution of `average_rating`

- **Question:** "Most books rate around 3.9. How wide is the spread?"
- **HINTS:**
  - Histogram with `bins=30`.
- **What you learn:** Skewed slightly to the high side. The mean is around 3.9.

Exploratory chart 2 — `num_pages` distribution

- **HINTS:**
  - Histogram with `bins=50`.
  - Set `range=(0, 1500)` to ignore extreme outliers.
- **What you learn:** Most books are 200-400 pages. Some are 0 (broken data!) or 5,000+ (encyclopedias?).

Exploratory chart 3 — `language_code` counts

- **HINTS:** `df['language_code'].value_counts().head(10).plot.bar()`.
- **What you learn:** Most books are "eng". A few are other languages. We will keep only English.

Phase B — Clean the books table (45 min)

Step 1 — Load CSV with `on_bad_lines='skip'`

- **WHAT:** Some rows have unescaped commas in the title. They break the CSV.
- **HINTS:**
  - `pd.read_csv('data/books.csv', on_bad_lines='skip')`.
- **EXPECTED:** ~11,000 rows. Maybe 5-10 bad rows are skipped.

Step 2 — Look at the DataFrame

- **WHAT:** Run `.info()`, `.head()`, `.shape`, `.dtypes`.
- **EXPECTED:** Some columns like `publication_date` are stored as text.

Step 3 — Parse `publication_date`

- **WHAT:** Convert the messy string ("9/16/2006") to datetime.
- **HINTS:**
  - `pd.to_datetime(df['publication_date'], errors='coerce', format='%m/%d/%Y')`.
  - About 1-2% will fail and become NaT. That is fine.

Step 4 — Find missing values

- **WHAT:** Run `df.isna().sum()`.
- **EXPECTED:** Few missing in most columns. `publication_date` may have some after parsing.

Step 5 — Drop books with `num_pages == 0` or NaN

- **WHAT:** A 0-page book is a data error.
- **HINTS:** `df = df[df['num_pages'] > 0].copy()`.

Step 6 — Filter to English books only

- **WHAT:** Keep only `language_code` starting with "en" (eng, en-US, en-GB).
- **HINTS:** `df = df[df['language_code'].str.startswith('en')].copy()`.
- **WHY:** Mixing languages hurts text features in Module 7.

Step 7 — Document everything

In a markdown cell, write: - Starting rows: ~11,000. - After dropping 0-page books: ~_____. - After English filter: ~_____. - WHY each decision.

Step 8 — Save to Drive

- **HINTS:** `df.to_parquet('/content/drive/MyDrive/goodreads_lab/books_step1.parquet')`.

Phase C — Make ONE chart for Ravi (15 min)

Ravi's chart — "Most books rate between 3.5 and 4.5"

A histogram of `average_rating` with vertical lines at 4.0 (his cutoff).

- **HINTS:**
  - `plt.hist(df['average_rating'], bins=40)`.
  - `plt.axvline(4.0, color='red', linestyle='--', label='Ravi cutoff')`.
- **Title:** "Goodreads rating distribution — only 35% of books exceed 4.0."
- **Takeaway:** "If we only publish books above 4.0, we still have many candidates. The cutoff is realistic."

Common mistakes in Class 1

Mistake	What goes wrong
Load without `on_bad_lines='skip'`	Code crashes on broken rows.
Forget `errors='coerce'` on date parse	Code crashes on one bad date.
Keep books with 0 pages	They are data errors and skew the model.
Keep all languages	Module 7 text features get confused.

Self-check before Class 2

- ☐ You loaded ~11,000 rows.
- ☐ `publication_date` is datetime.
- ☐ You dropped 0-page books.
- ☐ You filtered to English books.
- ☐ 3+ exploratory charts.

☐ 1 chart for Ravi.
☐ `books_step1.parquet` saved.

Class 2 — Encoding and Scaling

> **Scenario reminder:** Ravi says: “Some books have 1 author. Some have 5. The publisher column has 1,000 names. Some books got 5 ratings, some got 1 million. The model is a math model. Make this usable.”

Your goal

Parse author lists. Decide encoding for publisher. Log-transform skewed numerics.

Inputs

- `books_step1.parquet`

Outputs

- `books_step2.parquet` in Drive

Phase A — Explore the data first

Exploratory chart 1 — `ratings_count` distribution

- **HINTS:** Histogram. The distribution is very skewed.
- **What you learn:** Very long tail. Most books have <1000 ratings. A few have 1 million.

Exploratory chart 2 — `ratings_count_log` distribution

- **HINTS:** `np.log1p(ratings_count)` then histogram.
- **What you learn:** Log makes it much more normal.

Exploratory chart 3 — Top 20 publishers

- **HINTS:** `value_counts().head(20).plot.bar()`.
- **What you learn:** A few publishers (Penguin, Random House) dominate. Most are small.

Phase B — Encode and scale

Step 1 — Parse author lists

- **WHAT:** The `authors` column has multiple authors separated by `/`. Like “Author A/Author B”.
- **HINTS:**
  - `df['author_list'] = df['authors'].str.split('/')`.
  - `df['n_authors'] = df['author_list'].apply(len)`.
  - `df['primary_author'] = df['author_list'].apply(lambda x: x[0].strip())`.

Step 2 — Compute `primary_author_avg_rating`

- **WHAT:** For each primary author, compute the average book rating.
- **HINTS:**
  - `df.groupby('primary_author')['average_rating'].transform('mean')`.
- **WARNING:** This is leakage IF you use `average_rating` (the target). In M3 we accept it for EDA but in Class 4 we will be more careful.

Step 3 — One-hot encode top-5 language codes

- **WHAT:** Even after the English filter, there are sub-codes: "eng", "en-US", "en-GB".
- **HINTS:**
  - `pd.get_dummies(df, columns=['language_code'], prefix='lang')`.

Step 4 — Target-encode publisher

- **WHAT:** Publisher has ~1,000 unique values. Too many for one-hot.
- **HINTS:**
  - Use mean target encoding.
  - WARNING: only on train data in Class 4.
  - For now (EDA only): `df['publisher_mean_rating'] = df.groupby('publisher')['average_rating'].tra`

Step 5 — Log-transform skewed numerics

Column	New column
`num_pages`	`num_pages_log = np.log1p(num_pages)`
`ratings_count`	`ratings_count_log = np.log1p(ratings_count)`
`text_reviews_count`	`text_reviews_count_log = np.log1p(text_reviews_count)`

Step 6 — Save

- **HINTS:** `df.to_parquet('/content/drive/MyDrive/goodreads_lab/books_step2.parquet')`.

Phase C — Make ONE chart for Ravi

Ravi's chart — "Top 10 publishers by average rating"

A horizontal bar chart of the top 10 publishers by mean rating.

- **HINTS:**
  - Filter to publishers with >10 books.
  - GroupBy publisher, mean rating, sort, head 10.
- **Title:** "Top 10 publishers by average rating — small literary presses lead."
- **Takeaway:** "We should partner with the top 3 — they consistently publish well-rated books."

Common mistakes in Class 2

Mistake	What goes wrong
Target-encode publisher on FULL data	Leakage. Will look great in train. Bad in production.
One-hot encode publisher (~1000 values)	Table explodes to 1000 new columns.

Mistake	What goes wrong
Forget `np.log1p` and use `np.log` on zeros	`np.log(0) = -inf`. Crash.
Treat `primary_author` like a feature without target encoding	The model cannot handle 6,000 string values.

Self-check before Class 3

- ☐ `n_authors` and `primary_author` exist.
- ☐ `language_code` is encoded.
- ☐ `ratings_count_log`, `text_reviews_count_log`, `num_pages_log` exist.
- ☐ 3+ exploratory charts.
- ☐ 1 chart for Ravi.
- ☐ `books_step2.parquet` saved.

Class 3 — Feature Engineering

> **Scenario reminder:** Ravi says: "Now make me the SMART columns. How old is the book? Does the title have a colon (suggesting a subtitle, often academic)? How many ratings per text review (engagement signal)?"

Your goal

Engineer signals from dates, titles, and ratios.

Inputs

- `books_step2.parquet`

Outputs

- `books_step3.parquet` in Drive

Phase A — Explore the data first

Exploratory chart 1 — `book_age_years` distribution

- After Step 1 below.
- **HINTS:** Histogram.
- **What you learn:** Most books are recent (last 20 years). Some are 100+ years old (classics).

Exploratory chart 2 — `book_age_years` vs `average_rating`

- **HINTS:**
  - Bin `book_age_years` into 5 groups.
  - GroupBy bin, mean of `average_rating`.
  - Bar chart.
- **What you learn:** Older books tend to rate higher (survivor bias — only good old books are still in print).

Exploratory chart 3 — `n_authors` vs `average_rating`

- **HINTS:** Same pattern: groupby `n_authors`, mean rating, bar chart.
- **What you learn:** Single-author books tend to rate slightly higher than multi-author ones.

Phase B — Engineer the features

Step 1 — Date-derived features

New column	How to make it
`publication_year`	`df['publication_date'].dt.year`
`publication_month`	`.dt.month`
`book_age_years`	`2026 - publication_year`

Step 2 — Title features

New column	How to make it
`title_length`	`df['title'].str.len()`
`title_n_words`	`df['title'].str.split().apply(len)`
`has_subtitle`	`df['title'].str.contains(':').astype(int)`
`title_n_capital_words`	`df['title'].str.findall(r'\b[A-Z][a-z]+').apply(len)`

Step 3 — Engagement ratio features

New column	What it is
`ratings_per_text_review_ratio`	`ratings_count / (text_reviews_count + 1)`
`text_review_pct`	`text_reviews_count / (ratings_count + 1)`

- **WHY:** A book with many ratings but few text reviews = many people gave a star but did not bother to write. A book with high ratio of text reviews = many people wrote about it. Different engagement.

Step 4 — ISBN-13 features

- **WHAT:** `isbn13` is a 13-digit number. The first 3 digits are the country code.
- **HINTS:**
  - `df['isbn13_str'] = df['isbn13'].astype(str)`.
  - `df['isbn_first_3'] = df['isbn13_str'].str[:3]`.
- **WHY:** Different country codes = different publishing systems. A “978” prefix is global; “979” is newer.

Step 5 — Save

- **HINTS:** `df.to_parquet('/content/drive/MyDrive/goodreads_lab/books_step3.parquet')`.

Phase C — Make ONE chart for Ravi

Ravi’s chart — “Era vs average rating”

A line plot of average rating per decade.

- **HINTS:**
  - `df['decade'] = (df['publication_year'] // 10) * 10`.
  - `df.groupby('decade')['average_rating'].mean().plot()`.
- **Title:** "Older books rate higher — survivor bias in classic literature."
- **Takeaway:** "Do not be fooled. If we publish a NEW book, expect a rating closer to 3.8, not 4.2."

Common mistakes in Class 3

Mistake	What goes wrong
Use `2024` instead of current year for age	Age is wrong as time passes. Use `pd.Timestamp.now().year`.
Compute publisher_mean_rating on full data	Leakage.
Forget that ISBN is a number — apply string operations directly	Crashes. Cast to string first.

Self-check before Class 4

- ☐ Date features (year, month, age) exist.
- ☐ Title features (length, has_subtitle) exist.
- ☐ Engagement ratios exist.
- ☐ 3+ exploratory charts.
- ☐ 1 chart for Ravi.
- ☐ `books_step3.parquet` saved.

Class 4 — Feature Selection

> **Scenario reminder:** Ravi says: "You have ~25 columns. Pick the best 12-15. Justify each."

Your goal

Pick the best 12-15 columns. Justify every choice.

Inputs

- `books_step3.parquet`

Outputs

- `books_step4.parquet` in Drive
- A short markdown report explaining your choices

Phase A — Explore the data first

Exploratory chart 1 — Correlation heatmap of numeric columns

- **HINTS:**
  - `sns.heatmap(df[numeric_cols].corr(), annot=True, fmt='.2f')`.
- **What you learn:** Pairs with |corr| > 0.9 are redundant.

Exploratory chart 2 — Mutual information bars

- After Step 4 below.

Exploratory chart 3 — Random Forest importance bars

- After Step 5 below.

Phase B — Select features

Step 1 — Split into train and test FIRST

- **HINTS:**
  - `from sklearn.model_selection import train_test_split`.
  - `X = df.drop('average_rating', axis=1); y = df['average_rating']`.
  - `test_size=0.2`, `random_state=42`. NO stratify (regression).

Step 2 — Variance threshold

- **HINTS:**
  - `VarianceThreshold(threshold=0.01)`.

Step 3 — Correlation pruning

- **HINTS:**
  - Drop one of each pair with |corr| > 0.9.
  - Common candidates: `ratings_count` and `ratings_count_log` (keep only the log version).

Step 4 — Mutual information for regression

- **HINTS:**
  - `from sklearn.feature_selection import mutual_info_regression`.
  - `mi = mutual_info_regression(X_train[numeric_cols], y_train)`.
  - Sort and inspect.

Step 5 — Random Forest regressor importance

- **HINTS:**
  - `from sklearn.ensemble import RandomForestRegressor`.
  - `RandomForestRegressor(n_estimators=50, max_depth=8, n_jobs=-1)`.
  - `.fit(X_train, y_train)`.
  - `.feature_importances_`.

Step 6 — Pick the final 12-15 columns

- **WRITE DOWN:** In your notebook, list each column. Explain WHY.

Step 7 — Save

- Keep only selected columns + `average_rating`. Save as `books_step4.parquet`.

Phase C — Make ONE chart for Ravi

Ravi's chart — "Top 10 predictors of book rating"

A horizontal bar chart of the top 10 features.

- **Title:** "Top 10 predictors of book rating."
- **Takeaway:** "ratings_count, page count, and publisher dominate. Description text adds smaller but useful signal."

Common mistakes in Class 4

Mistake	What goes wrong
Use `mutual_info_classif` (target is continuous)	Wrong function. Use `mutual_info_regression`.
Forget to drop `ratings_count_log` AND `ratings_count`	Both highly correlated. Keep only one.
Stratify split for regression	`stratify=y` does not work for continuous target.

Self-check before Class 5

- ☐ Train/test split done FIRST.
- ☐ 12-15 columns remain + `average_rating`.
- ☐ You wrote WHY for each kept column.
- ☐ 3+ exploratory charts.
- ☐ 1 chart for Ravi.
- ☐ `books_step4.parquet` saved.

Class 5 — Pipelines

> **Scenario reminder:** Ravi says: "Your code is in 4 notebooks. When a new manuscript arrives, will you copy 4 notebooks to the server? No. We need ONE Pipeline."

Your goal

Build ONE Pipeline that takes a raw book row and predicts the rating.

Inputs

- `books_step4.parquet`

Outputs

- `goodreads_pipeline.joblib` in Drive

Phase A — Explore the data first

Exploratory chart 1 — Residual plot

- After training, plot `(y_test - y_pred)` vs `y_pred`.
- **HINTS:** `plt.scatter(y_pred, y_test - y_pred)`.

- **What you learn:** Where the model is bad.

Exploratory chart 2 — Predicted vs actual scatter

- **HINTS:** `plt.scatter(y_test, y_pred)` with a diagonal line.
- **What you learn:** Close to diagonal = good predictions.

Phase B — Build the pipeline

Step 1 — Decide numeric vs categorical columns

- **EXAMPLE:**
  - numeric: `num_pages_log`, `ratings_count_log`, `text_reviews_count_log`, `book_age_years`, `n_authors`, `title_length`, `has_subtitle`.
  - categorical: `language_code`, `isbn_first_3` (if kept).

Step 2 — Numeric mini-pipeline

- **HINTS:**
  - Skeleton:
    ```
    numeric_transformer = Pipeline(steps=[
        ('imputer', SimpleImputer(strategy='___')),
        ('scaler',  StandardScaler()),
    ])
    ```
  - Fill in: `'median'`.

Step 3 — Categorical mini-pipeline

- **HINTS:**
  - Skeleton:
    ```
    categorical_transformer = Pipeline(steps=[
        ('imputer', SimpleImputer(strategy='___')),
        ('onehot',  OneHotEncoder(handle_unknown='___', sparse_output=___)),
    ])
    ```
  - Fill in: `'most_frequent'`, `'ignore'`, `False`.

Step 4 — ColumnTransformer

- **HINTS:**
  - `ColumnTransformer(transformers=[('num', ..., numeric_cols), ('cat', ..., categorical_cols)])`.

Step 5 — Add the model

- **HINTS:**
  - For REGRESSION, use `Ridge(alpha=1.0, random_state=42)` or `LinearRegression()`.
- **WHY Ridge instead of LinearRegression?** Ridge handles collinearity better.

Step 6 — Train and evaluate

- **HINTS:**
  - `full_pipeline.fit(X_train, y_train)`.
  - `from sklearn.metrics import mean_absolute_error`.
  - `mae = mean_absolute_error(y_test, full_pipeline.predict(X_test))`.
- **EXPECTED:** MAE around 0.25-0.35 stars.

Step 7 — Save the trained pipeline

- **HINTS:**
  - `joblib.dump(full_pipeline, '/content/drive/MyDrive/goodreads_lab/goodreads_pipeline.joblib`

Phase C — Make ONE chart for Ravi

Ravi's chart — "MAE of 0.3 stars means we are usually within half a star"

A simple bar chart showing distribution of absolute errors.

- **HINTS:**
  - `errors = abs(y_test - y_pred)`.
  - Histogram of errors.
- **Title:** "Prediction errors — 80% within 0.5 stars of the true rating."
- **Takeaway:** "Good enough to decide between 'publish' (predict > 4.0) and 'reject'."

Common mistakes in Class 5

Mistake	What goes wrong
Use a classifier (e.g., LogisticRegression) for regression	Wrong tool. Use regressor.
Forget `sparse_output=False`	OneHotEncoder returns sparse matrix. Some code breaks.
Forget `random_state`	Results change every run.

Self-check before Class 6

- ☐ Pipeline has preprocessor + regressor.
- ☐ Numeric + categorical transformers in preprocessor.
- ☐ `handle_unknown='ignore'` set.
- ☐ MAE under 0.40.
- ☐ Residual + predicted-vs-actual charts.
- ☐ `goodreads_pipeline.joblib` saved.

Class 6 — End-to-End Lab (THE BIG ONE)

> **Scenario reminder:** Ravi is in the meeting room. He wants the FINAL clean dataset on his desk in 90 minutes.

Your goal

Take the raw CSV. Produce ONE final `.parquet` file with the exact 21-column schema. Plus a 1-page findings report.

Time

90 minutes of focused work.

Outputs

- `goodreads_books_clean.parquet` (~10,500 rows × 21 columns) in Drive
- `findings.md` (~1 page)
- Your Colab notebook

Phase A — Explore the data first (10 minutes)

Reuse 3 of your best charts: 1. `average_rating` distribution with the 4.0 cutoff line. 2. Era vs rating (survivor bias). 3. Top 10 most important features.

Phase B — Build the final dataset (70 minutes)

Required output schema

#	Column	Type	Source
1	`bookID`	int	raw
2	`title_length`	int	engineered
3	`title_n_words`	int	engineered
4	`has_subtitle`	int (0/1)	engineered
5	`n_authors`	int	engineered
6	`primary_author`	string	engineered
7	`primary_author_avg_rating`	float	engineered (train-only)
8	`language_code`	string	raw (filtered to en*)
9	`num_pages_log`	float	engineered
10	`ratings_count_log`	float	engineered
11	`text_reviews_count_log`	float	engineered
12	`ratings_per_text_review_ratio`	float	engineered
13	`isbn13`	string	raw
14	`publisher_target_encoded`	float	engineered (train-only)
15	`publication_year`	int	engineered
16	`publication_month`	int	engineered
17	`book_age_years`	int	engineered
18	`description_length`	int	engineered (if description joined)
19	`n_named_entities`	int	engineered (spaCy on description, optional)
20	`average_rating`	float	TARGET (raw)
21	`description`	string	kept for Module 7 (if joined)

Lab phases (timed)

Phase	Time	What you do
1. Load	10 min	Load with `on_bad_lines='skip'`.
2. Clean	10 min	Parse dates, drop 0-page, filter English.
3. Encode	15 min	Parse authors, language one-hot, log-transform.

Phase	Time	What you do
4. Engineer	25 min	Date features, title features, ratios, ISBN parsing, target encoding (train-only).
5. Validate + save	10 min	Check 21 columns + dtypes. Save `.parquet`.
6. Findings	10 min	Write `findings.md`.

Total: 90 minutes.

Phase C — Findings report for Ravi (10 minutes)

Write `findings.md` with these sections:

Final numbers

- Final row count: ____
- Average rating: ____ (should be ~3.9)
- % above 4.0: ____% (Ravi's cutoff)

Top 3 insights

1. _____
2. _____
3. _____

One question to investigate in Module 4

- _____

One chart that summarizes everything

Embed your most important chart.

Grading rubric (100 points)

Item	Points
All 21 columns exist with the right names and dtypes	25
Cleaning decisions written in markdown	10
Pipeline reproducible (one command, raw CSV to `.parquet`)	15
Target encoding done on TRAIN ONLY (no leakage)	10
Filter to English documented	5
At least 3 exploratory + 1 explanatory chart per class	**15**
`findings.md` has insights, not just numbers	10
Code is clean	10

Submit

Upload to `module-3/class_6/submissions/<YourTeamName>/`: - `goodreads_books_clean.parquet` - Your Colab notebook - `findings.md`

Tips for the whole module

1. **Do NOT copy-paste code from this guide.** Write the code yourself.
2. **Save your notebook to Drive often.** Colab disconnects after 12 hours.
3. **Save intermediate `.parquet` files to Drive.**
4. **Pair-program.** One student types, one reads. Switch every 20 minutes.
5. **Make a chart BEFORE you write cleaning code.** See the problem first.
6. **Make a chart AFTER each step.** Confirm your code did what you expected.
7. **Ask the mentor early.** If stuck for 20 minutes, ask.

Good luck.